Projeto da UC de Inteligência Artificial

Alunos:
Danilo Issamu Santos Takata - 202012561
Deiner Luiz Oliveira Junior - 202217864
Hendrwey Mederios Nogueira - 201911868
Pedro Henrique Guimarães Rezende - 201910463

Descrição do Tema:

O projeto apresentado tem o objetivo de criar uma Rede Neural Convolucional para que ela possa reconhecer determinadas palavras dentro de um conjunto de outras.

Preparação Inicial:

Durante o desenvolvimento do projeto foi utilizada algumas ferramentas bem importantes e necessárias para que o sistema funcione de maneira plena, como:

- Pycharm;
- Python 3.10.4;
- Bibliotecas Tensorflow e Keras;
- Bibliotecas Os e Pathlib;
- Bibliotecas Matplotlib e Seaborn;
- Biblioteca Numpy;

Importação das bibliotecas e Carregamento da Base de dados:

```python
import os
import pathlib

import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import tensorflow as tf

from keras import layers
from keras import models

seed = 42
tf.random.set_seed(seed)
np.random.seed(seed)

DATASET_PATH = 'data/mini_speech_commands'

data_dir = pathlib.Path(DATASET_PATH)
if not data_dir.exists():
    tf.keras.utils.get_file(
        'mini_speech_commands.zip',
        origin="http://storage.googleapis.com/download.tensorflow.org/data/mini_speech_commands.zip",
        extract=True,
        cache_dir='.', cache_subdir='data')

commands = np.array(tf.io.gfile.listdir(str(data_dir)))
commands = commands[commands != 'README.md']
print('Comandos:', commands)
```

Nessa parte o sistema importa as bibliotecas necessárias para poder funcionar, acontece logo em seguida a setagem do valor ‘semente’ para o experimento de reproducibilidade. A terceira parte dessa parte do código é a averiguação da base de dados do sistema, caso ele não exista o próprio sistema realiza o download da base de áudios na URI passada na variável origin, se existir o código pula essa etapa. Na parte final da imagem é apresentado o trecho do código responsável em criar um array unidimensional tendo como elementos os comando que serão analisados e utilizados durante o funcionamento da rede neural.

```
Comandos: ['down' 'go' 'left' 'no' 'right' 'stop' 'up' 'yes']
```

A imagem acima mostra no terminal do sistema o array criado com os respectivos comandos que serão trabalhados no sistema.

/==========/

```
filenames = tf.io.gfile.glob(str(data_dir) + '/*/*')
filenames = tf.random.shuffle(filenames)
num_samples = len(filenames)
print('Número de exemplos:', num_samples)
print('Número de exemplos por comando:',
      len(tf.io.gfile.listdir(str(data_dir / commands[0]))))
print('Exemplo de arquivo Tensor:', filenames[0])

train_files = filenames[:6400]
val_files = filenames[6400: 6400 + 800]
test_files = filenames[-700:]

print('Tamanho do Conjunto de Treino', len(train_files))
print('Tamanho do Conjunto de Validação', len(val_files))
print('Tamanho do Conjunto de Teste', len(test_files))
```

A parte do código apresentada é responsável por resgatar na base de áudios uma lista de áudios e embaralhar, logo em seguida apresentar a quantidade de áudios na base por inteiro, por comando trabalhado e um exemplo de tensor utilizado no código. Na parte seguinte a base é splitada (segmentada ou dividida) em partes de tamanhos e seções diferentes (treino, validação e teste).

```
Número de exemplos: 8000
Número de exemplos por comando: 1000
Exemplo de arquivo Tensor: tf.Tensor(b'data\\mini_speech_commands\\down\\b83c1acf_nohash_0.wav', shape=(), dtype=string)
Tamanho do Conjunto de Treino 6400
Tamanho do Conjunto de Validação 800
Tamanho do Conjunto de Teste 700
```

As printagens do código acima no terminal do sistema.
/==========/

Leitura dos arquivos de áudio e seus respectivos rótulos:

```python
test_file = tf.io.read_file(DATASET_PATH + '/down/0a9f9af7_nohash_0.wav')
test_audio, _ = tf.audio.decode_wav(contents=test_file)
var = test_audio.shape


def decode_audio(audio_binary):
    audio, _ = tf.audio.decode_wav(contents=audio_binary)
    return tf.squeeze(audio, axis=-1)


def get_label(file_path):
    parts = tf.strings.split(
        input=file_path,
        sep=os.path.sep)
    return parts[-2]


def get_waveform_and_label(file_path):
    label = get_label(file_path)
    audio_binary = tf.io.read_file(file_path)
    waveform = decode_audio(audio_binary)
    return waveform, label
```

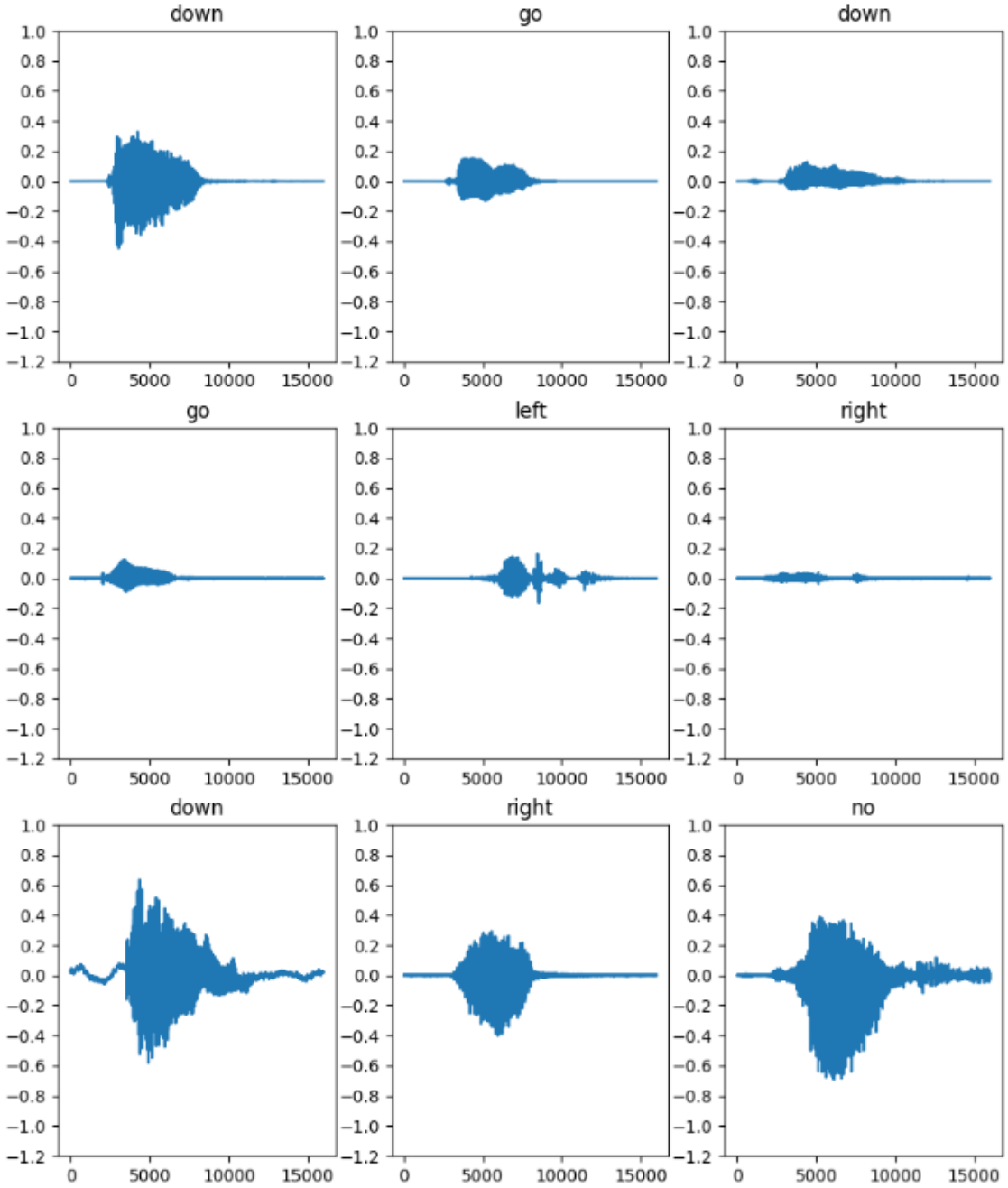
O código pega um áudio na base de dados e começar a analisar e extrair seus dados para retornar no terminal alguns dados do mesmo, como sua amplitude e se ele é um arquivo de som mono ou estéreo. Essa imagem também mostra a responsabilidade de criar funções que irão extrair mais informações dos áudios trabalhados.

- decode_audio é uma função responsável por receber um arquivo de áudio de realizar o pré-processamento dele, ou seja, padronizá-lo.
- get_label é outra função que irá criar os rótulos de cada áudio utilizando o diretório pai encontrado no sistema.
- get_waveform_and_label é a função auxiliar que irá receber o rótulo do áudio e devolver uma tupla contendo os tensores de áudio e rótulo prontos para aprendizado supervisionado.

/==========/

```
AUTOTUNE = tf.data.AUTOTUNE

files_ds = tf.data.Dataset.from_tensor_slices(train_files)

waveform_ds = files_ds.map(
    map_func=get_waveform_and_label,
    num_parallel_calls=AUTOTUNE)

rows = 3
cols = 3
n = rows * cols
fig, axes = plt.subplots(rows, cols, figsize=(10, 12))

for i, (audio, label) in enumerate(waveform_ds.take(n)):
    r = i // cols
    c = i % cols
    ax = axes[r][c]
    ax.plot(audio.numpy())
    ax.set_yticks(np.arange(-1.2, 1.2, 0.2))
    label = label.numpy().decode('utf-8')
    ax.set_title(label)

plt.show()
```

Nessa parte do código irão ser criadas os conjuntos de validação e de teste, juntamente com a criação das formas de algumas ondas de alguns áudios.



Resultado da última printagem do código.
/==========/

Transformando Ondas de áudio em Espectrogramas:

As formas de onda no conjunto de dados são representadas no domínio do tempo. É preciso transformar as formas de onda dos sinais no domínio do tempo em sinais no domínio da frequência do tempo calculando a transformada de Fourier de curta duração (STFT) para converter as formas de onda em espectrogramas , que mostram mudanças de frequência ao longo do tempo e podem ser representados como imagens 2D.

Uma transformada de Fourier (tf.signal.fft) converte um sinal para suas frequências componentes, mas perde todas as informações de tempo. Em comparação, STFT (tf.signal.stft) divide o sinal em janelas de tempo e executa uma transformada de Fourier em cada janela, preservando algumas informações de tempo e retornando um tensor 2D no qual você pode executar convoluções padrão.

```
def get_spectrogram(waveform):
    input_len = 16000
    waveform = waveform[:input_len]
    zero_padding = tf.zeros(
        [16000] - tf.shape(waveform),
        dtype=tf.float32)
    waveform = tf.cast(waveform, dtype=tf.float32)
    equal_length = tf.concat([waveform, zero_padding], 0)
    spectrogram = tf.signal.stft(
        equal_length, frame_length=255, frame_step=128)
    spectrogram = tf.abs(spectrogram)
    spectrogram = spectrogram[..., tf.newaxis]
    return spectrogram


for waveform, label in waveform_ds.take(1):
    label = label.numpy().decode('utf-8')
    spectrogram = get_spectrogram(waveform)

print('Label:', label)
print('Formato da Onda:', waveform.shape)
print('Formato do Espectrograma:', spectrogram.shape)
```

- get_spectogram é a função responsável por receber a onda do áudio em questão e fazer com que ele se torna em um espectrograma, por meio principalmente do tf.zero (zera os áudios menores que 1 segundo), tf.abs (trabalha com a magnitude do áudio) e tf.signal.stft (transforma a onda em espectrograma por meio de parâmetros).

Depois disso, é devolvido alguns dados do áudio tensorizado trabalhado.

```
Label: down
Formato da Onda: (16000,)
Formato do Espectrograma: (124, 129, 1)
```

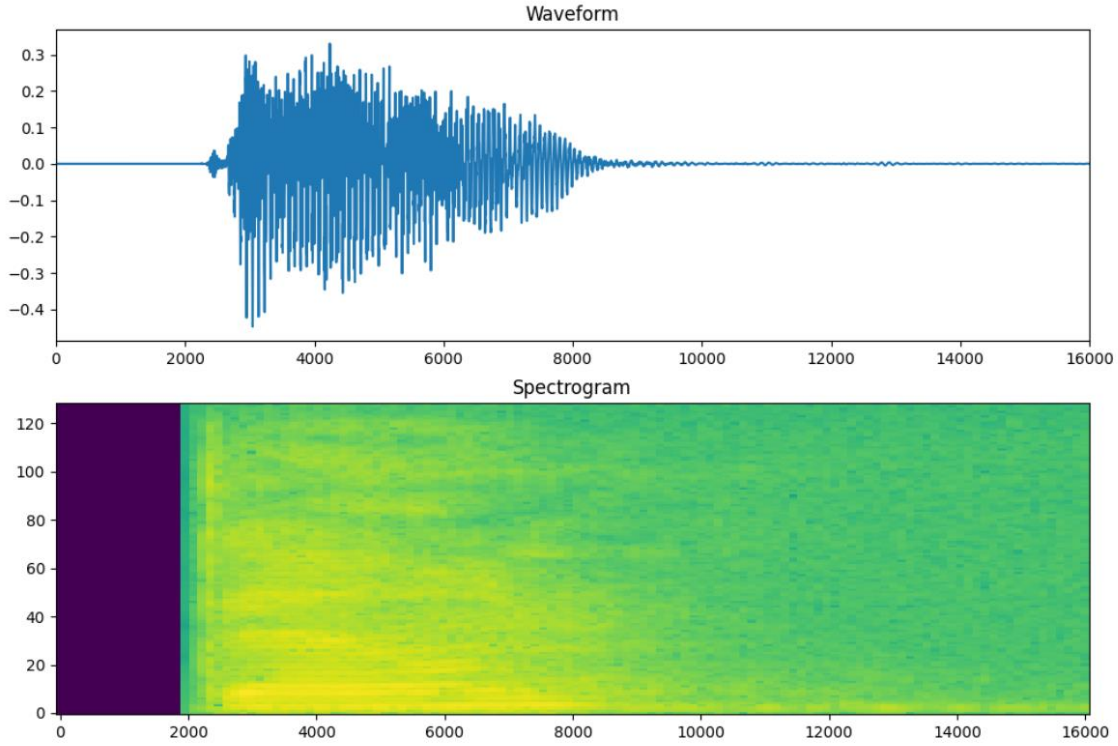
Resultado no terminal do print acima.

/==========/

```
def plot_spectrogram(spectrogram, ax):
    if len(spectrogram.shape) > 2:
        assert len(spectrogram.shape) == 3
    spectrogram = np.squeeze(spectrogram, axis=-1)
    log_spec = np.log(spectrogram.T + np.finfo(float).eps)
    height = log_spec.shape[0]
    width = log_spec.shape[1]
    X = np.linspace(0, np.size(spectrogram), num=width, dtype=int)
    Y = range(height)
    ax.pcolormesh(X, Y, log_spec)


fig, axes = plt.subplots(2, figsize=(12, 8))
timescale = np.arange(waveform.shape[0])
axes[0].plot(timescale, waveform.numpy())
axes[0].set_title('Waveform')
axes[0].set_xlim([0, 16000])

plot_spectrogram(spectrogram.numpy(), axes[1])
axes[1].set_title('Spectrogram')
plt.show()
```

- plot_spectrogram é a função que irá plotar o espectrograma.

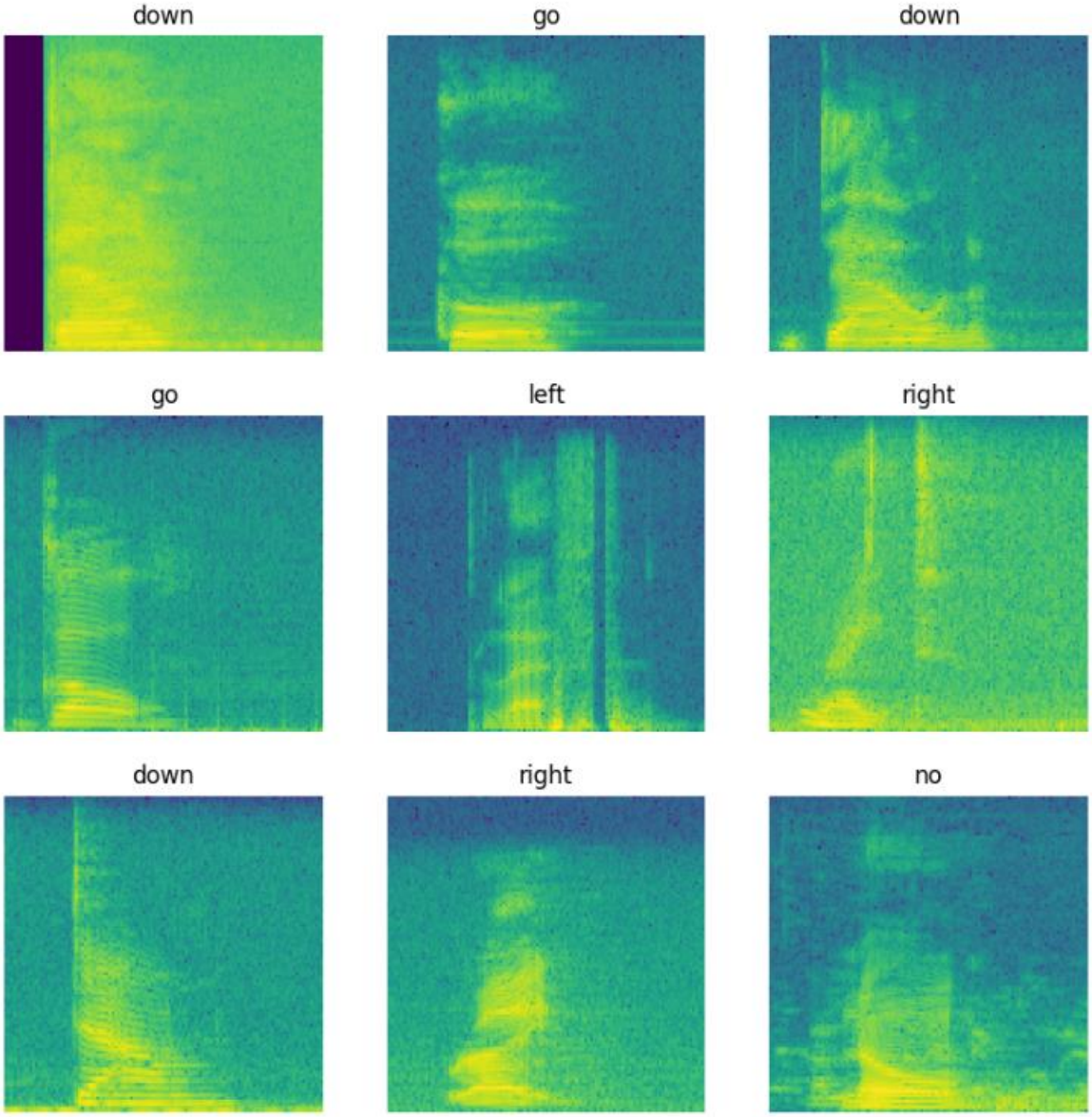
  Depois disso, é construído o gráfico do espectrograma utilizando a função anterior.


Waveform
0.3
0.2
0.1
0.0
−0.1
−0.2
−0.3
−0.4
0
2000
4000
6000
8000
10000
12000
14000
16000
Spectrogram
120
100
80
60
40
20
0
0
2000
4000
6000
8000
10000
12000
14000
16000

/==========/

```python
def get_spectrogram_and_label_id(audio, label):
    spectrogram = get_spectrogram(audio)
    label_id = tf.argmax(label == commands)
    return spectrogram, label_id


spectrogram_ds = waveform_ds.map(
    map_func=get_spectrogram_and_label_id,
    num_parallel_calls=AUTOTUNE)

rows = 3
cols = 3
n = rows * cols
fig, axes = plt.subplots(rows, cols, figsize=(10, 10))

for i, (spectrogram, label_id) in enumerate(spectrogram_ds.take(n)):
    r = i // cols
    c = i % cols
    ax = axes[r][c]
    plot_spectrogram(spectrogram.numpy(), ax)
    ax.set_title(commands[label_id.numpy()])
    ax.axis('off')

plt.show()
```

Esse pedaço do código é a parte que transforma as ondas de áudio em espectrogramas com seus respectivos rótulos, logo em seguida realiza a plotagem deles.



A plotagem dos áudios trabalhados anteriormente.
/==========/

Construção e Treino do modelo:

```python
def preprocess_dataset(files):
    files_ds = tf.data.Dataset.from_tensor_slices(files)
    output_ds = files_ds.map(
        map_func=get_waveform_and_label,
        num_parallel_calls=AUTOTUNE)
    output_ds = output_ds.map(
        map_func=get_spectrogram_and_label_id,
        num_parallel_calls=AUTOTUNE)
    return output_ds


train_ds = spectrogram_ds
val_ds = preprocess_dataset(val_files)
test_ds = preprocess_dataset(test_files)

batch_size = 64
train_ds = train_ds.batch(batch_size)
val_ds = val_ds.batch(batch_size)

train_ds = train_ds.cache().prefetch(AUTOTUNE)
val_ds = val_ds.cache().prefetch(AUTOTUNE)

for spectrogram, _ in spectrogram_ds.take(1):
    input_shape = spectrogram.shape
print('Input shape:', input_shape)
num_labels = len(commands)

norm_layer = layers.Normalization()
norm_layer.adapt(data=spectrogram_ds.map(map_func=lambda spec, label: spec))
```

Nessa parte o processo de pré-processamento das bases de validação e teste são repetidos para poderem serem inseridos na rede neural, essas bases serão juntadas para o treinamento do modelo.

As operações Dataset.cache e Dataset.prefetch para reduzir a latência de leitura durante o treinamento do modelo.

```
model = models.Sequential([
    layers.Input(shape=input_shape),
    layers.Resizing(32, 32),
    norm_layer,
    layers.Conv2D(32, 3, activation='relu'),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_labels),
])

model.summary()

model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=['accuracy'],
)
```

Para o modelo,usará uma rede neural convolucional simples (CNN), pois transformou os arquivos de áudio em imagens de espectrograma.

O modelo tf.keras.Sequential usará as seguintes camadas de pré-processamento Keras:

- tf.keras.layers.Resizing : para reduzir a amostra da entrada para permitir que o modelo treine mais rápido.
- tf.keras.layers.Normalization : para normalizar cada pixel da imagem com base em sua média e desvio padrão.

Para a camada de Normalization, o método de adapt precisaria primeiro ser chamado nos dados de treinamento para calcular estatísticas agregadas (ou seja, a média e o desvio padrão).

Usa o modelo Keras com o otimizador Adam e a perda de entropia cruzada.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 resizing (Resizing)         (None, 32, 32, 1)         0

 normalization (Normalizatio  (None, 32, 32, 1)        3
 n)

 conv2d (Conv2D)             (None, 30, 30, 32)        320

 conv2d_1 (Conv2D)           (None, 28, 28, 64)        18496

 max_pooling2d (MaxPooling2D  (None, 14, 14, 64)       0
 )

 dropout (Dropout)           (None, 14, 14, 64)        0

 flatten (Flatten)           (None, 12544)             0

 dense (Dense)               (None, 128)               1605760

 dropout_1 (Dropout)         (None, 128)               0

 dense_1 (Dense)             (None, 8)                 1032

=================================================================
```

```
Total params: 1,625,611
Trainable params: 1,625,608
Non-trainable params: 3
```

Dados do modelo criado nas imagens do código anteriormente.
/==========/

```
EPOCHS = 10

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=tf.keras.callbacks.EarlyStopping(verbose=1, patience=2),
)
```

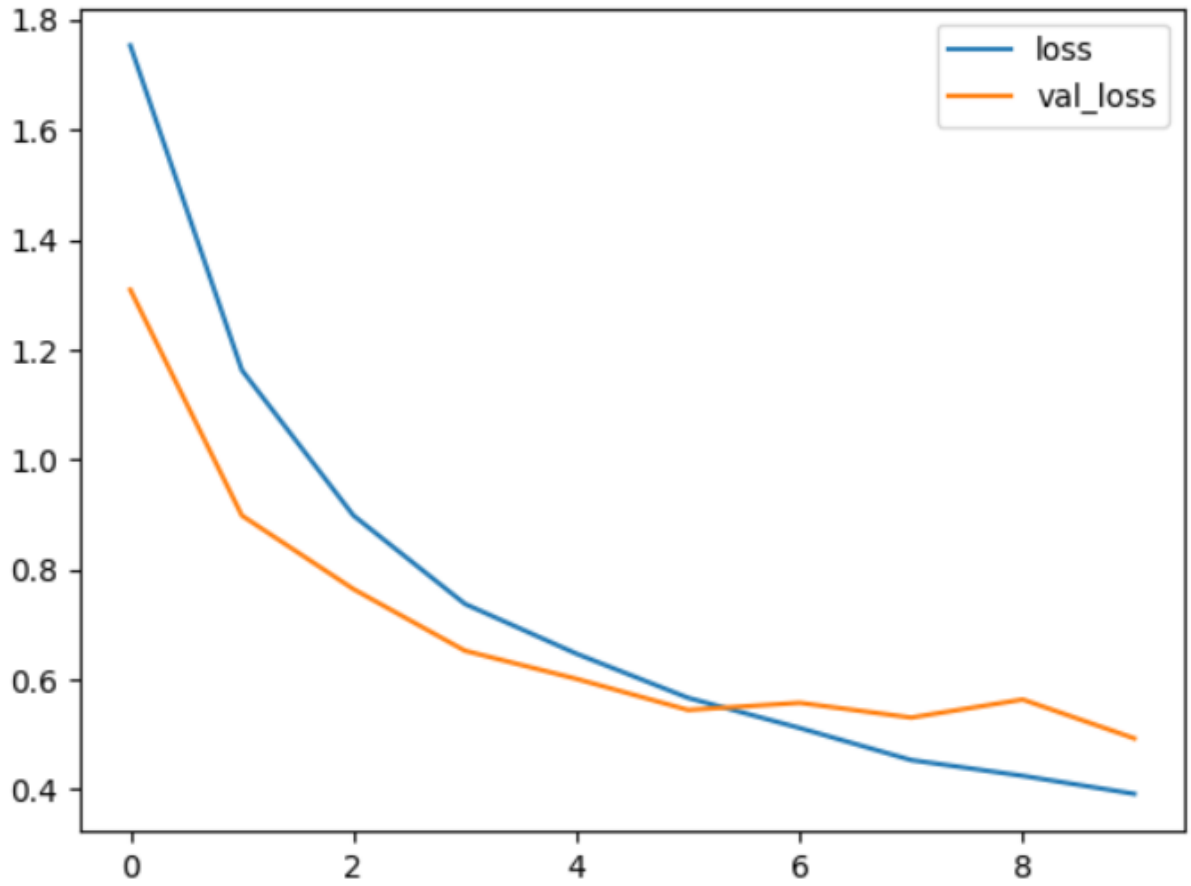
O treino do modelo será realizado em 10 épocas.

```
Epoch 1/10
100/100 [==============================] - 45s 437ms/step - loss: 1.7537 - accuracy: 0.3692 - val_loss: 1.3093 - val_accuracy: 0.5763
Epoch 2/10
100/100 [==============================] - 14s 140ms/step - loss: 1.1631 - accuracy: 0.5850 - val_loss: 0.8985 - val_accuracy: 0.6988
Epoch 3/10
100/100 [==============================] - 13s 132ms/step - loss: 0.8985 - accuracy: 0.6806 - val_loss: 0.7647 - val_accuracy: 0.7475
Epoch 4/10
100/100 [==============================] - 13s 130ms/step - loss: 0.7376 - accuracy: 0.7366 - val_loss: 0.6524 - val_accuracy: 0.7700
Epoch 5/10
100/100 [==============================] - 13s 128ms/step - loss: 0.6470 - accuracy: 0.7658 - val_loss: 0.6012 - val_accuracy: 0.7850
Epoch 6/10
100/100 [==============================] - 13s 129ms/step - loss: 0.5667 - accuracy: 0.7992 - val_loss: 0.5446 - val_accuracy: 0.8087
Epoch 7/10
100/100 [==============================] - 13s 128ms/step - loss: 0.5118 - accuracy: 0.8177 - val_loss: 0.5576 - val_accuracy: 0.8062
Epoch 8/10
100/100 [==============================] - 13s 130ms/step - loss: 0.4535 - accuracy: 0.8388 - val_loss: 0.5308 - val_accuracy: 0.8200
Epoch 9/10
100/100 [==============================] - 13s 130ms/step - loss: 0.4248 - accuracy: 0.8428 - val_loss: 0.5636 - val_accuracy: 0.8163
Epoch 10/10
100/100 [==============================] - 14s 135ms/step - loss: 0.3920 - accuracy: 0.8631 - val_loss: 0.4930 - val_accuracy: 0.8275
22/22 [==============================] - 1s 20ms/step
```

Dados de cada época realizada pelo sistema durante o funcionamento.
/==========/

```
metrics = history.history
plt.plot(history.epoch, metrics['loss'], metrics['val_loss'])
plt.legend(['loss', 'val_loss'])
plt.show()
```

Codificação da construção da curva de Aprendizado.



/==========/

Avaliação do modelo:

```
test_audio = []
test_labels = []

for audio, label in test_ds:
  test_audio.append(audio.numpy())
  test_labels.append(label.numpy())

test_audio = np.array(test_audio)
test_labels = np.array(test_labels)

y_pred = np.argmax(model.predict(test_audio), axis=1)
y_true = test_labels

test_acc = sum(y_pred == y_true) / len(y_true)
print(f'Acurácia: {test_acc:.0%}')
```

Execução do modelo no conjunto de teste e verifique o desempenho do modelo.
Código responsável por analisar a acurácia do sistema.

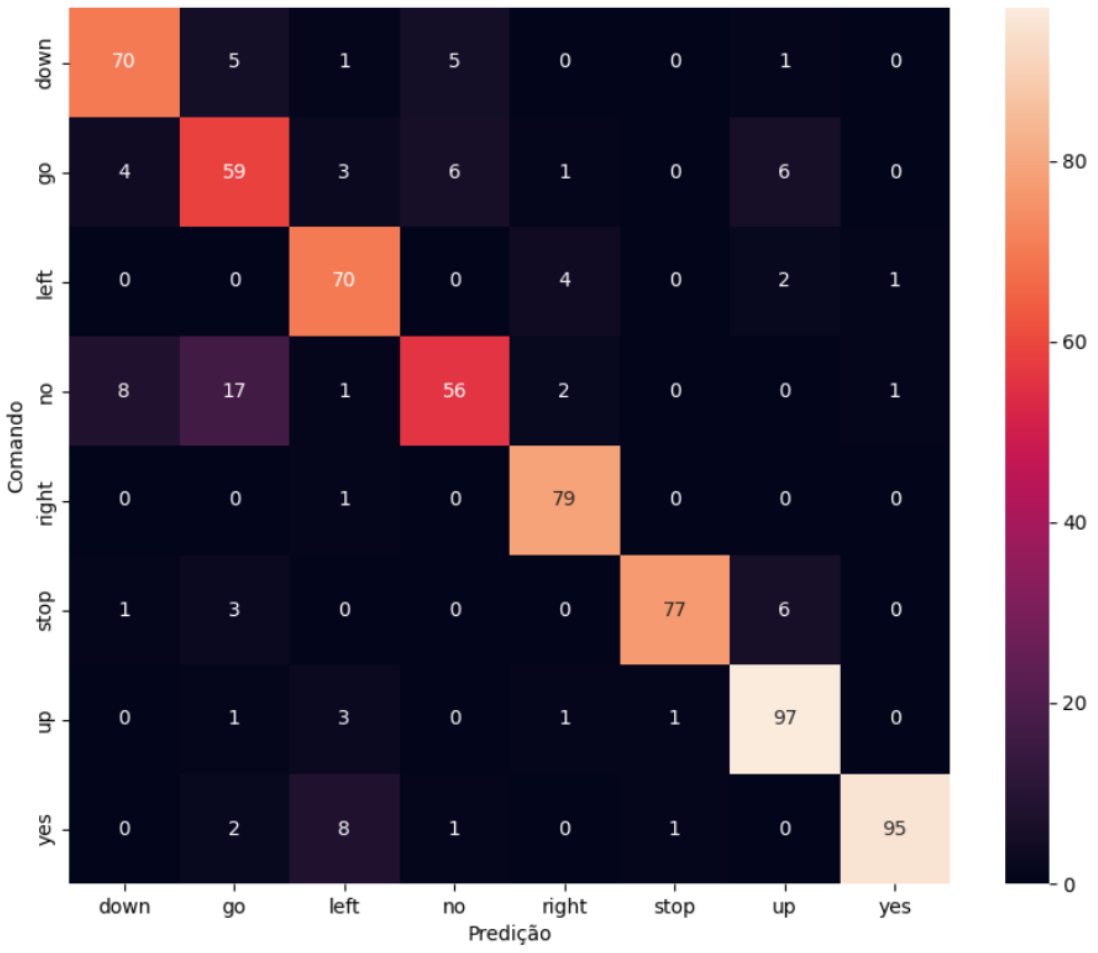
```
Acurácia: 86%
```

Resultado da acurácia.

/==========/

```
confusion_mtx = tf.math.confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_mtx,
            xticklabels=commands,
            yticklabels=commands,
            annot=True, fmt='g')
plt.xlabel('Predição')
plt.ylabel('Comando')
plt.show()
```

É construído uma matriz de confusão para verificar quão bem o modelo classificou cada um dos comandos no conjunto de teste.

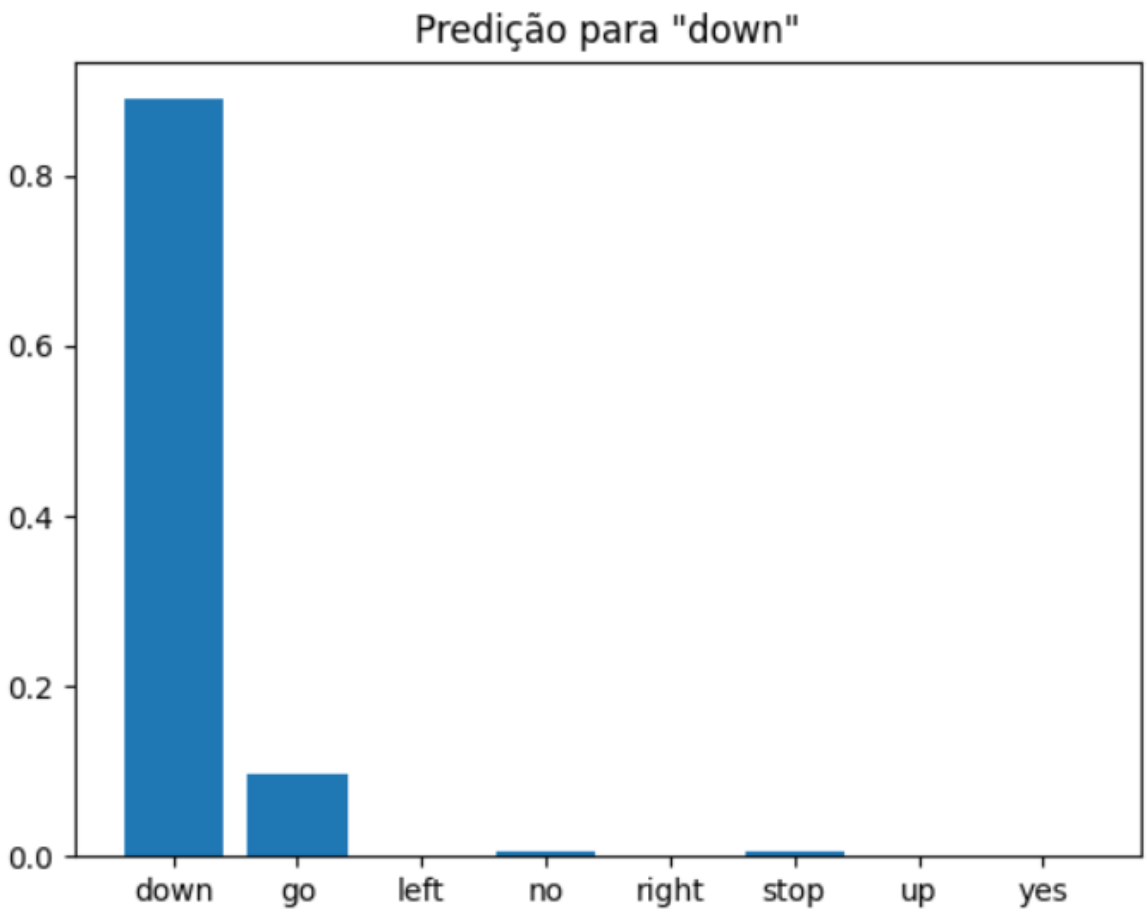


/==========/

Predição do Sistema:

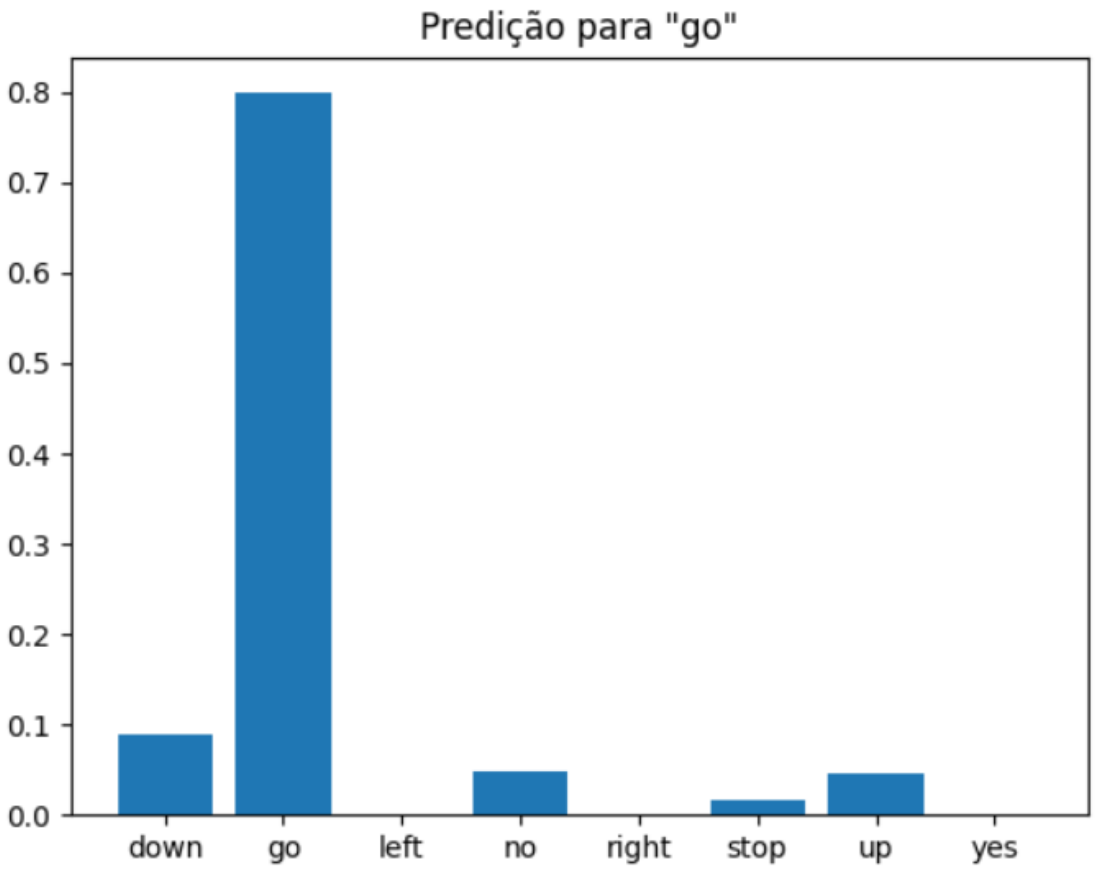
Essa parte é onde o sistema será colocado em funcionamento, irão ser passados nos códigos a seguir o caminho de alguns áudios que foram separados intencionalmente para que eles possam ser utilizados como parâmetros para a rede neural realizar a identificação.

```
sample_fileD = data_dir/'down/1a4259c3_nohash_0.wav'

sample_ds = preprocess_dataset([str(sample_fileD)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```



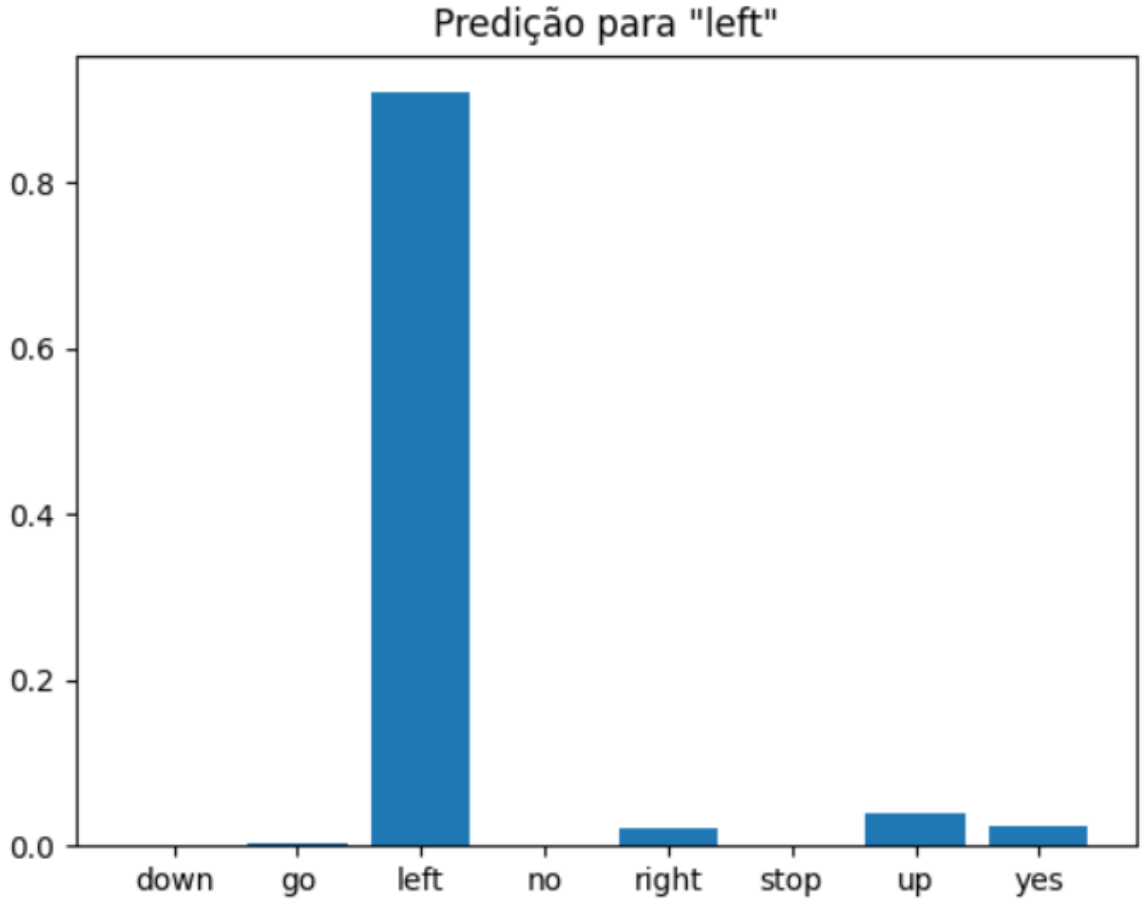
/==========/

```
sample_fileG = data_dir/'go/ffd2ba2f_nohash_2.wav'

sample_ds = preprocess_dataset([str(sample_fileG)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```



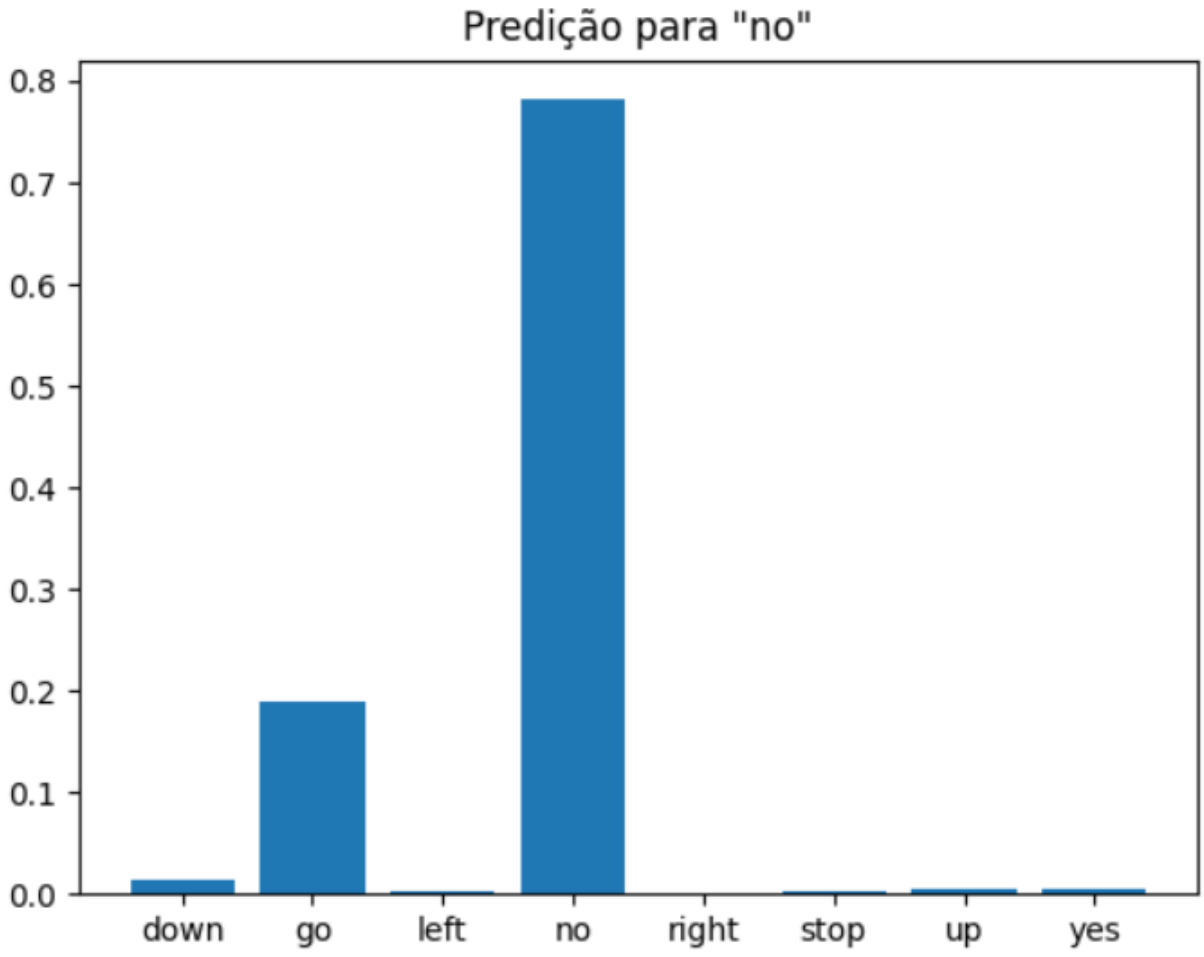
/==========/

```
sample_fileL = data_dir/'left/fb2f3242_nohash_0.wav'

sample_ds = preprocess_dataset([str(sample_fileL)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```



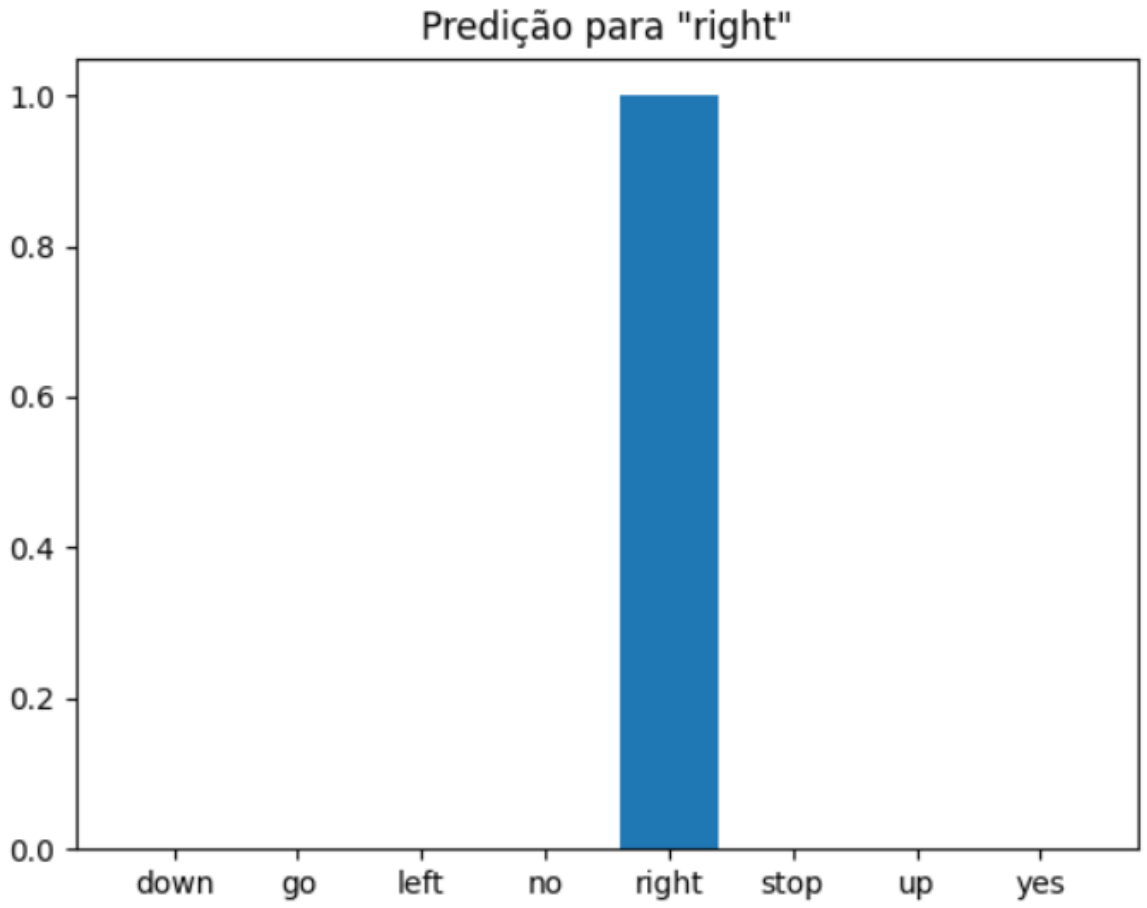
/==========/

```
sample_fileN = data_dir/'no/ffd2ba2f_nohash_1.wav'

sample_ds = preprocess_dataset([str(sample_fileN)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```



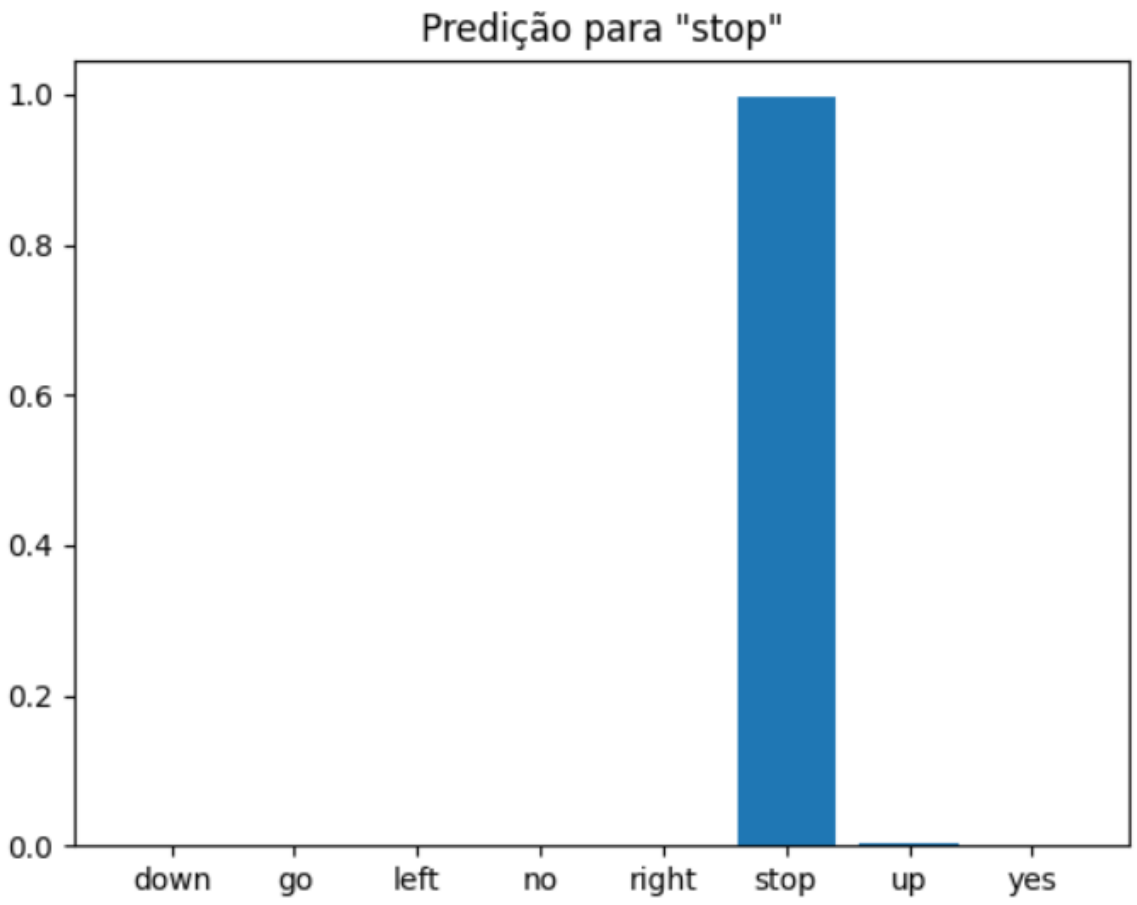
/==========/

```
sample_fileR = data_dir/'right/0c40e715_nohash_1.wav'

sample_ds = preprocess_dataset([str(sample_fileR)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```



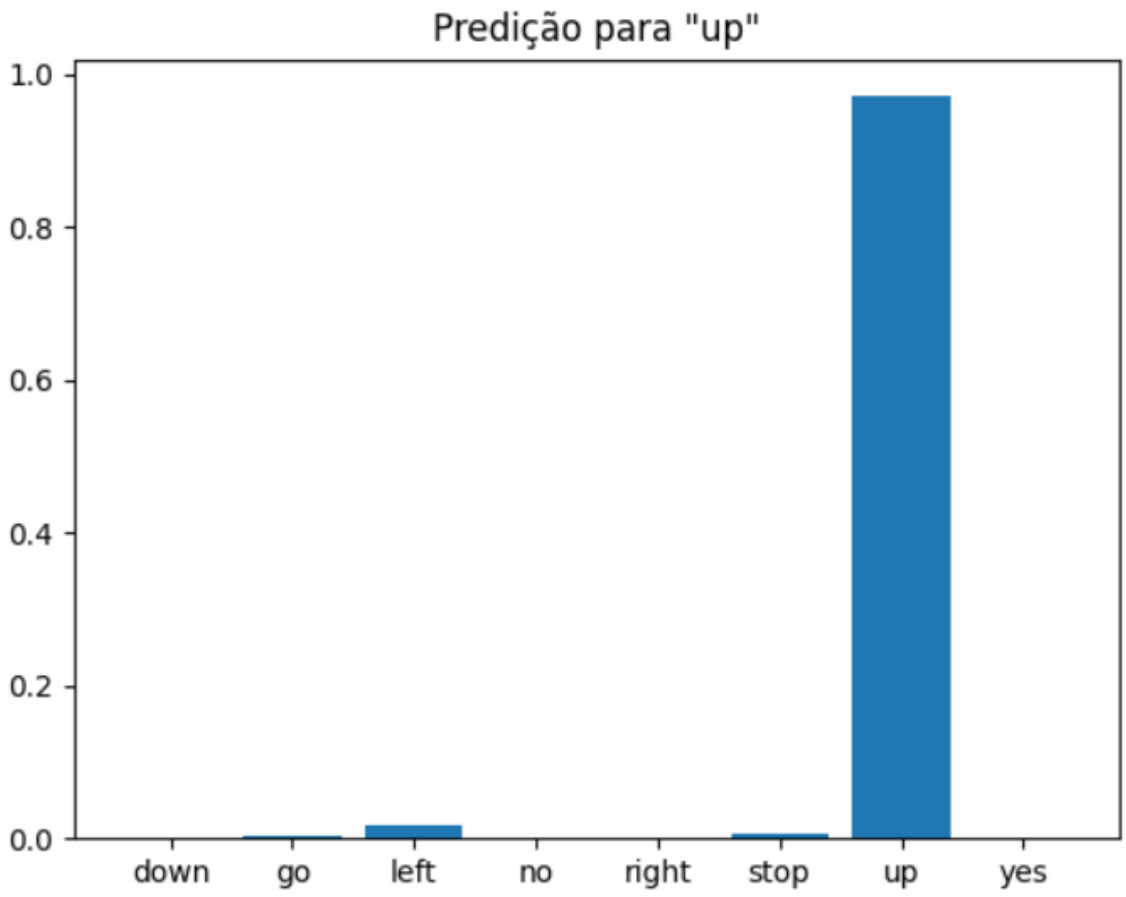
/==========/

```
sample_fileS = data_dir/'stop/1a892463_nohash_0.wav'

sample_ds = preprocess_dataset([str(sample_fileS)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```



/==========/

```
sample_fileU = data_dir/'up/1a073312_nohash_0.wav'

sample_ds = preprocess_dataset([str(sample_fileU)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```



/==========/

```
sample_fileY = data_dir/'yes/1ecfb537_nohash_4.wav'

sample_ds = preprocess_dataset([str(sample_fileY)])

for spectrogram, label in sample_ds.batch(1):
    prediction = model(spectrogram)
    plt.bar(commands, tf.nn.softmax(prediction[0]))
    plt.title(f'Predição para "{commands[label[0]]}"')
    plt.show()
```

Predição para "yes"
0.0
0.2
0.4
0.6
0.8
down
go
left
no
right
stop
up
yes